



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
з дисципліни «Розробка мобільних застосунків під Android»
Тема: «ДОСЛІДЖЕННЯ РОБОТИ З ВБУДОВАНИМИ ДАТЧИКАМИ»
Варіант 10

Виконала:

студентка групи ІА-23

Єрмак Д. Р

Перевірив:

Орленко С. П.

Мета роботи: ознайомитись з можливостями вбудованих датчиків мобільних пристроїв та дослідити способи їх використання для збору та обробки даних.

Завдання на лабораторну роботу

Написати програму під платформу Андроїд, яка має інтерфейс для виведення даних з обраного вбудованого датчика (тип обирається самостійно, можна відслідковувати зміни значень і з декількох датчиків).

Функціональність базового додатку додатково розширюється обробкою отриманих даних та виведенням їх у відповідній формі.

- крокомір (підрахунок кількості кроків);

Хід роботи

1. В класі MainActivity пишемо такий код:

```
package com.example.lab1_android

import android.content.Context
import android.content.Intent
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import android.util.Log
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import kotlin.math.absoluteValue
import kotlin.math.sqrt

class MainActivity : AppCompatActivity(), SensorEventListener {
    private lateinit var sensorManager: SensorManager
    private var accelerometerSensor: Sensor? = null
    private lateinit var stepCountText: TextView
    private lateinit var startTrainingButton: Button
    private lateinit var stopTrainingButton: Button
    private lateinit var manualStepInput: EditText
    private lateinit var addManualStepsButton: Button
    private lateinit var resetStepsButton: Button
    private lateinit var viewStatsButton: Button
    private val viewModel: StepCounterViewModel by viewModels()
    private val threshold = 0.5
    private val minStepInterval = 250L
    private val alpha = 0.8

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        stepCountText = findViewById(R.id.step_count_text)
        startTrainingButton = findViewById(R.id.start_training_button)
        stopTrainingButton = findViewById(R.id.stop_training_button)
        manualStepInput = findViewById(R.id.manual_step_input)
        addManualStepsButton = findViewById(R.id.add_manual_steps_button)
        resetStepsButton = findViewById(R.id.reset_steps_button)
```

```

        viewStatsButton = findViewById(R.id.view_stats_button)

        viewModel.initialize(this)

        sensorManager = getSystemService(Context.SENSOR_SERVICE) as
SensorManager
        accelerometerSensor =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

        if (accelerometerSensor == null) {
            stepCountText.text = "Акселерометр не підтримується на цьому
пристрої"
        } else {
            stepCountText.text = "Кількість кроків:
${viewModel.stepCount}"
        }

        startTrainingButton.setOnClickListener {
            viewModel.startTraining()
            startTrainingButton.isEnabled = false
            stopTrainingButton.isEnabled = true
            stepCountText.text = "Кількість кроків:
${viewModel.stepCount}"
        }

        stopTrainingButton.setOnClickListener {
            viewModel.stopTraining(this)
            startTrainingButton.isEnabled = true
            stopTrainingButton.isEnabled = false
        }

        addManualStepsButton.setOnClickListener {
            val steps = manualStepInput.text.toString().toIntOrNull() ?: 0
            if (steps > 0) {
                viewModel.addManualSteps(steps)
                stepCountText.text = "Кількість кроків:
${viewModel.stepCount}"
                manualStepInput.text.clear()
            }
        }

        resetStepsButton.setOnClickListener {
            viewModel.resetSteps()
            stepCountText.text = "Кількість кроків:
${viewModel.stepCount}"
        }

        viewStatsButton.setOnClickListener {
            val intent = Intent(this, StatisticsActivity::class.java)
            intent.putParcelableArrayListExtra("training_sessions",
ArrayList(viewModel.getTrainingSessions()))
            startActivity(intent)
        }

        override fun onResume() {
            super.onResume()
            accelerometerSensor?.also { sensor ->
                sensorManager.registerListener(this, sensor,
SensorManager.SENSOR_DELAY_FASTEST)
            }
        }

        override fun onPause() {
            super.onPause()
            sensorManager.unregisterListener(this)
        }

```

```

    }

    override fun onSensorChanged(event: SensorEvent?) {
        event?.let {
            if (it.sensor.type == Sensor.TYPE_ACCELEROMETER &&
                viewModel.isTrainingActive) {
                viewModel.gravity[0] = (alpha * viewModel.gravity[0] + (1
                - alpha) * it.values[0]).toFloat()
                viewModel.gravity[1] = (alpha * viewModel.gravity[1] + (1
                - alpha) * it.values[1]).toFloat()
                viewModel.gravity[2] = (alpha * viewModel.gravity[2] + (1
                - alpha) * it.values[2]).toFloat()

                viewModel.linearAcceleration[0] = it.values[0] -
                viewModel.gravity[0]
                viewModel.linearAcceleration[1] = it.values[1] -
                viewModel.gravity[1]
                viewModel.linearAcceleration[2] = it.values[2] -
                viewModel.gravity[2]

                val magnitude = sqrt(
                    (viewModel.linearAcceleration[0] *
                    viewModel.linearAcceleration[0] +
                    viewModel.linearAcceleration[1] *
                    viewModel.linearAcceleration[1] +
                    viewModel.linearAcceleration[2] *
                    viewModel.linearAcceleration[2]).toDouble()
                )

                Log.d("StepCounter", "Magnitude: $magnitude, Diff:
                ${ (magnitude - viewModel.lastMagnitude).absoluteValue}")

                val currentTime = System.currentTimeMillis()
                if (viewModel.lastMagnitude > 0 && (magnitude -
                viewModel.lastMagnitude).absoluteValue > threshold &&
                    (currentTime - viewModel.lastStepTime) >
                    minStepInterval) {
                    viewModel.stepCount++
                    viewModel.lastStepTime = currentTime
                    stepCountText.text = "Кількість кроків:
                    ${viewModel.stepCount}"
                    Log.d("StepCounter", "Step detected! Count:
                    ${viewModel.stepCount}")
                }
                viewModel.lastMagnitude = magnitude
            }
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}

```

2. В activity_main.xml також додаємо код:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/step_count_text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Кількість кроків: 0"
    >

```

```
        android:textSize="24sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="32dp"/>
```

```
<Button
    android:id="@+id/start_training_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Почати тренування"
    app:layout_constraintTop_toBottomOf="@id/step_count_text"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp"/>
```

```
<Button
    android:id="@+id/stop_training_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Завершити тренування"
    android:enabled="false"
    app:layout_constraintTop_toBottomOf="@id/start_training_button"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp"/>
```

```
<EditText
    android:id="@+id/manual_step_input"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:hint="Введіть кількість кроків"
    android:inputType="number"
    app:layout_constraintTop_toBottomOf="@id/stop_training_button"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp"/>
```

```
<Button
    android:id="@+id/add_manual_steps_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Додати кроки вручну"
    app:layout_constraintTop_toBottomOf="@id/manual_step_input"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp"/>
```

```
<Button
    android:id="@+id/reset_steps_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Скинути лічильник"
    app:layout_constraintTop_toBottomOf="@id/add_manual_steps_button"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp"/>
```

```
<Button
    android:id="@+id/view_stats_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Переглянути статистику"
    app:layout_constraintTop_toBottomOf="@id/reset_steps_button"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
```

```

        android:layout_marginTop="16dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>

```

3. В StatisticsActivity додаємо такий код:

```

package com.example.lab1_android

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView

class StatisticsActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_statistics)

        val recyclerView: RecyclerView =
            findViewById(R.id.training_sessions_recycler_view)
        recyclerView.layoutManager = LinearLayoutManager(this)

        val sessions =
            intent.getParcelableArrayListExtra<TrainingSession>("training_sessions")
            ?: arrayListOf()
        recyclerView.adapter = TrainingSessionAdapter(sessions)

        val backButton: Button = findViewById(R.id.back_button)
        backButton.setOnClickListener {
            finish()
        }
    }
}

class TrainingSessionAdapter(private val sessions: List<TrainingSession>)
:
    RecyclerView.Adapter<TrainingSessionAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val sessionInfo: TextView = view.findViewById(android.R.id.text1)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(android.R.layout.simple_list_item_1, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val session = sessions[position]
        val durationMinutes = session.duration / 1000 / 60
        val durationSeconds = (session.duration / 1000) % 60
        holder.sessionInfo.text = "Тренування ${position + 1}
        (${session.timestamp}): ${session.steps} кроків, " +
            "Тривалість: $durationMinutes хв $durationSeconds сек"
    }

    override fun getItemCount(): Int = sessions.size
}

```

4. В activity_statistics.xml додаємо такий код:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/stats_title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Статистика тренувань"
        android:textSize="24sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp"/>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/training_sessions_recycler_view"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toBottomOf="@id/stats_title"
        app:layout_constraintBottom_toTopOf="@id/back_button"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp"
        android:layout_marginStart="16dp"
        android:layout_marginEnd="16dp"
        android:layout_marginBottom="16dp"/>

    <Button
        android:id="@+id/back_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Назад"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginBottom="16dp"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

5. В StepCounterViewModel додаємо такий код:

```
package com.example.lab1_android

import android.content.Context
import android.os.Parcel
import android.os.Parcelable
import androidx.lifecycle.ViewModel
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

data class TrainingSession(
    val steps: Int,
    val duration: Long,
    val timestamp: String
) : Parcelable {
    constructor(parcel: Parcel) : this(
        steps = parcel.readInt(),
        duration = parcel.readLong(),
        timestamp = parcel.readString() ?: ""
    )
}
```

```

        override fun writeToParcel(parcel: Parcel, flags: Int) {
            parcel.writeInt(steps)
            parcel.writeLong(duration)
            parcel.writeString(timestamp)
        }

        override fun describeContents(): Int = 0

        companion object CREATOR : Parcelable.Creator<TrainingSession> {
            override fun createFromParcel(parcel: Parcel): TrainingSession {
                return TrainingSession(parcel)
            }

            override fun newArray(size: Int): Array<TrainingSession?> {
                return arrayOfNulls(size)
            }

            fun fromJson(json: String): TrainingSession {
                val parts = json.split("|")
                return TrainingSession(parts[0].toInt(), parts[1].toLong(),
parts[2])
            }

            fun toJson(session: TrainingSession): String {
                return
"$${session.steps}|${session.duration}|${session.timestamp}"
            }
        }
    }

class StepCounterViewModel : ViewModel() {
    var stepCount: Int = 0
    var lastMagnitude: Double = 0.0
    var lastStepTime: Long = 0L
    var gravity: FloatArray = FloatArray(3) { 0f }
    var linearAcceleration: FloatArray = FloatArray(3) { 0f }

    var isTrainingActive: Boolean = false
    private var trainingStartTime: Long = 0L
    private val trainingSessions = mutableListOf<TrainingSession>()

    fun initialize(context: Context) {
        val prefs = context.getSharedPreferences("StepCounterPrefs",
Context.MODE_PRIVATE)
        val sessionsJson = prefs.getString("training_sessions", null)
        if (sessionsJson != null) {
            trainingSessions.clear()
            sessionsJson.split(";").filter { it.isNotEmpty() }.forEach {
                trainingSessions.add(TrainingSession.fromJson(it))
            }
        }
    }

    fun startTraining() {
        isTrainingActive = true
        trainingStartTime = System.currentTimeMillis()
        stepCount = 0
    }

    fun stopTraining(context: Context) {
        if (isTrainingActive) {
            isTrainingActive = false
            val duration = System.currentTimeMillis() - trainingStartTime
            val timestamp = SimpleDateFormat("dd/MM/yyyy HH:mm:ss",
Locale.getDefault()).format(Date())
            val session = TrainingSession(stepCount, duration, timestamp)

```



```

        trainingSessions.add(session)
        saveSessions(context)
    }

    fun addManualSteps(steps: Int) {
        stepCount += steps
    }

    fun resetSteps() {
        stepCount = 0
    }

    fun getTrainingSessions(): List<TrainingSession> {
        return trainingSessions.toList()
    }

    private fun saveSessions(context: Context) {
        val prefs = context.getSharedPreferences("StepCounterPrefs",
Context.MODE_PRIVATE)
        val sessionsJson = trainingSessions.joinToString(";") {
TrainingSession.toJson(it) }
        with(prefs.edit()) {
            putString("training_sessions", sessionsJson)
            apply()
        }
    }
}

```

6. В AndroidManifest.xml додаємо такий код:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.lab1_android">

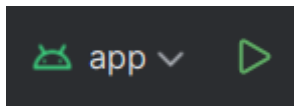
    <uses-permission
android:name="android.permission.ACTIVITY_RECOGNITION" />
    <uses-permission
android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.NoActionBar">
        <activity android:name=".StatisticsActivity" />
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER"
/>
            </intent-filter>
        </activity>
    </application>
</manifest>

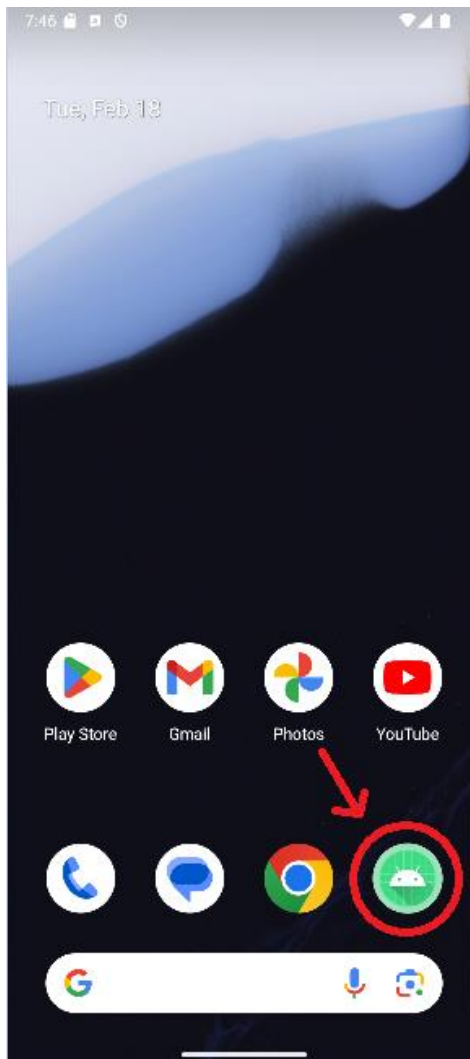
```

Середовище розробки Android Studio надає певний емулятор смартфона з ОС Android 15.0, а саме Medium Phone API 35, скористаємось ним для перегляду результату виконання програми.

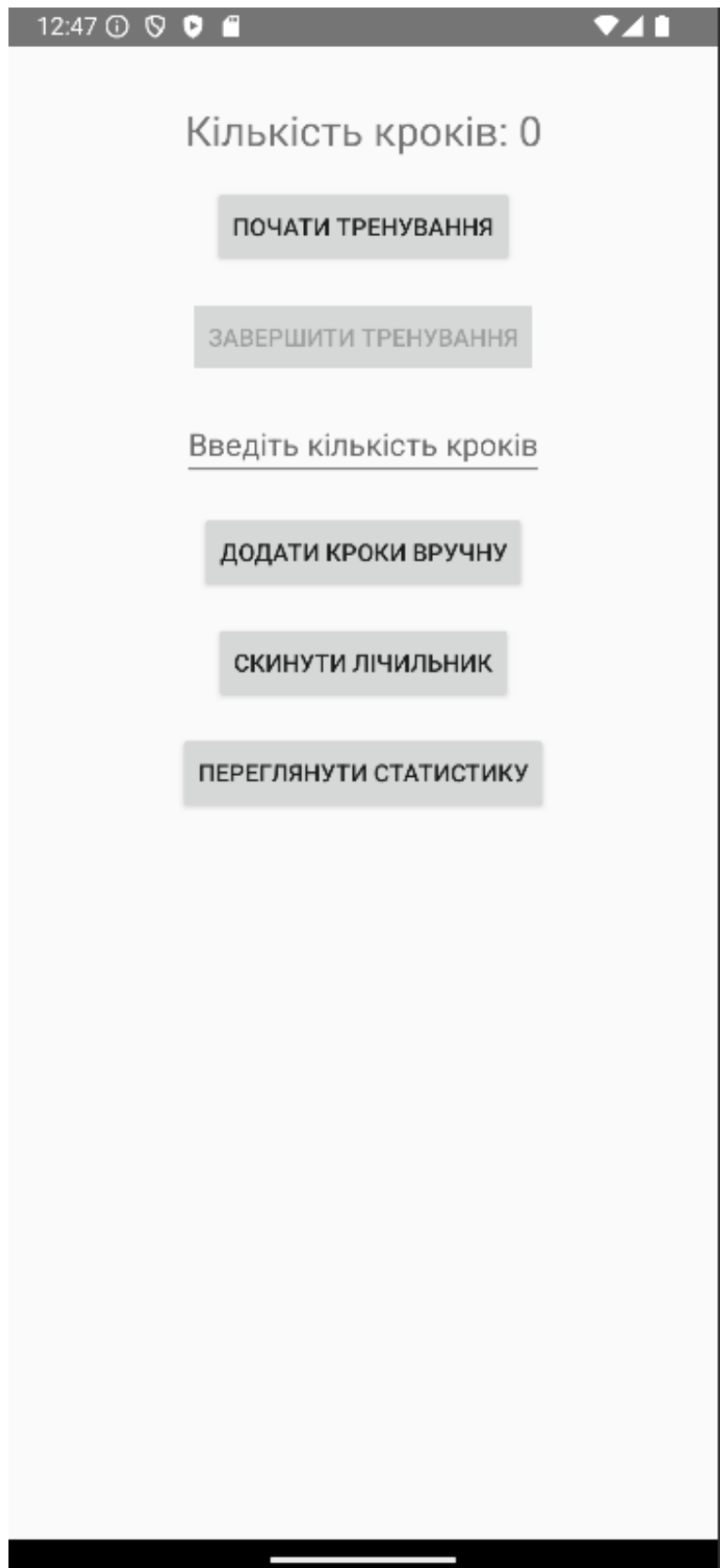
Запускаємо застосунок, чекаємо білд програми, і отримуємо результат:



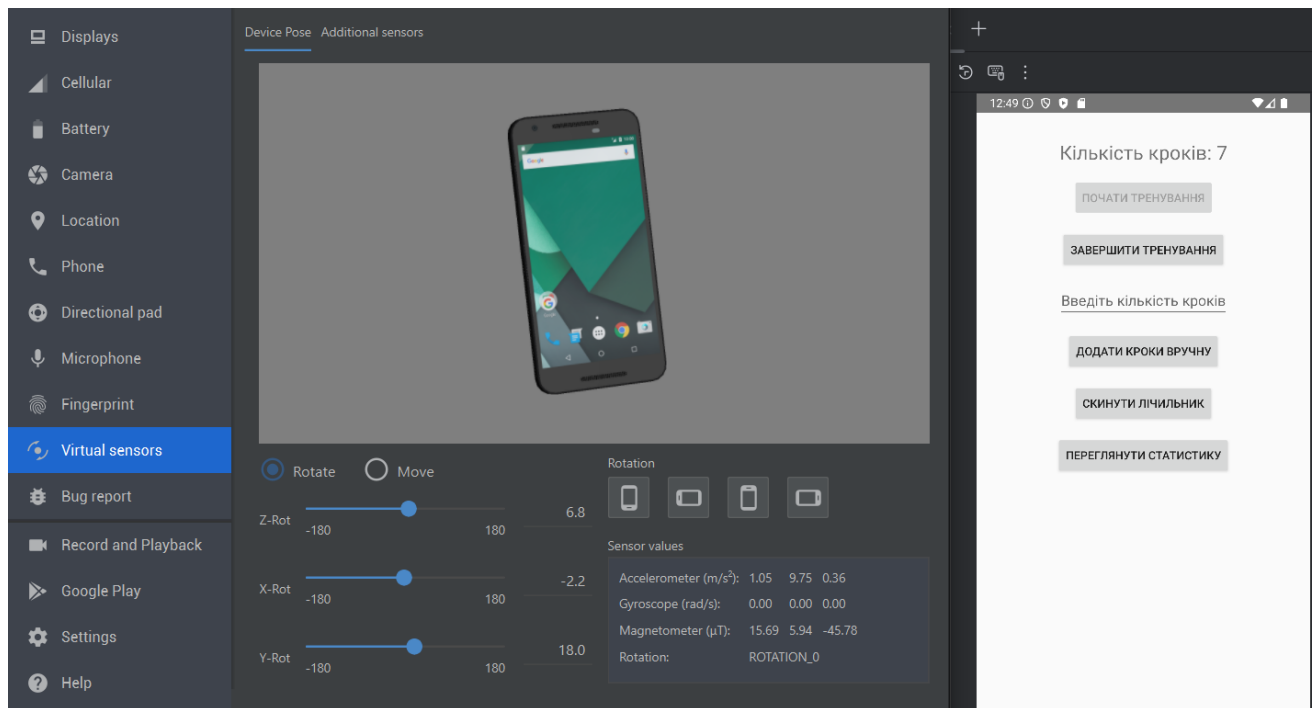
На робочому столі емулятора з'являється іконка програми:



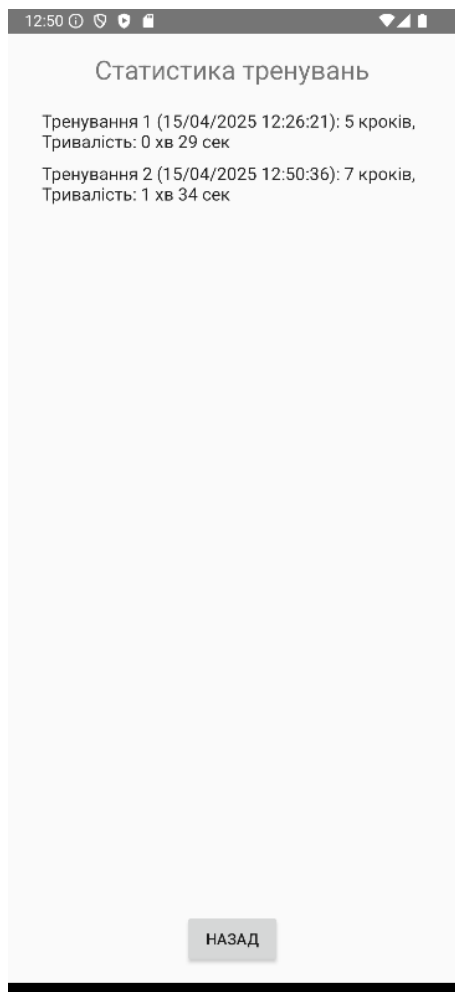
Відкриваємо програму і бачимо нашу робочу програму:



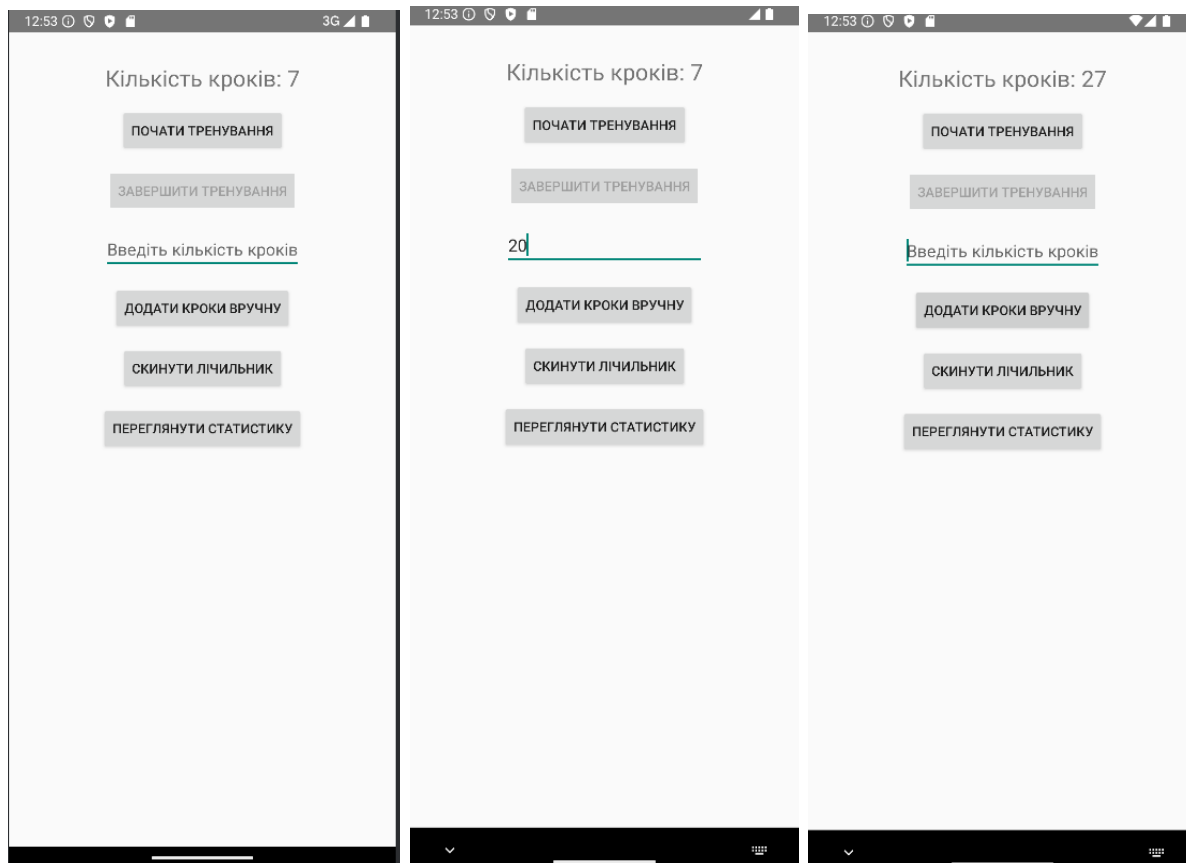
Ми бачимо кнопки для початку та закінчення тренування, додавання кроків вручну, перегляду тренувань та скидання лічильнику кроків.



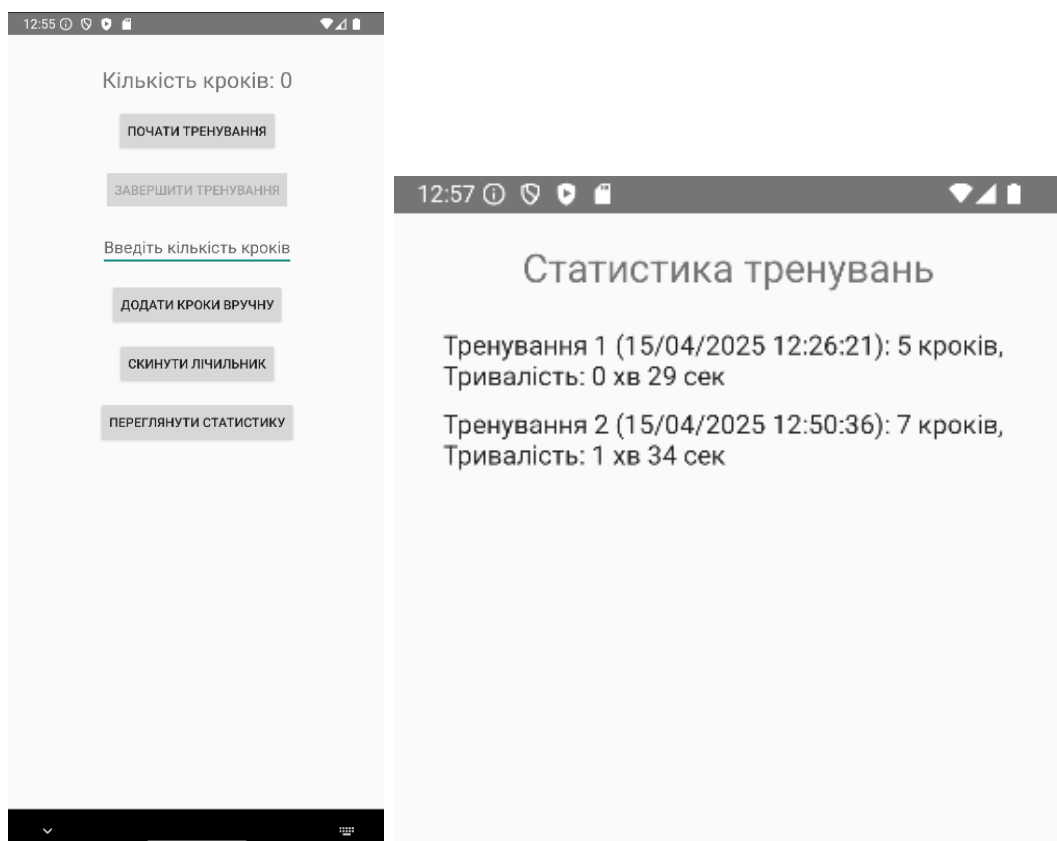
Натискаємо кнопку початку тренування, рухаємо телефон за допомогою Virtual sensors та бачимо збільшення кількості кроків



Можемо закінчити тренування та перейти до перегляду тренувань, натиснувши кнопку «Переглянути статистику» та повернутись на головну сторінку через кнопку «Назад»



Можемо додавати кроки вручну, записавши кількість та натиснувши кнопку «Додати кроки вручну»



Також можемо скинути лічильник кроків, натиснувши на кнопку «Скинути лічильник», при тому статистика лишається збереженою, обнуляється лише лічильник

Отже, застосунок працює відповідно до вимог завдання.

Висновок: У ході виконання даної лабораторної роботи було досліджено можливості вбудованих датчиків мобільних пристроїв та способи їх використання для збору та обробки даних